

Boolean Algebra And Logic Circuit

12 September 2026 09:58

The binary operations performed by any digital circuit with the set of elements 0 and 1 are called logical operations or logical functions. The algebra used to symbolically represents the logic function called Boolean algebra. It is a two state algebra invented by a mathematician George Boole in 1854.

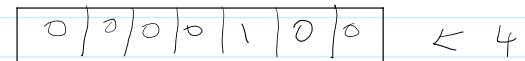
Thus, the Boolean algebra is a system of mathematical logic for analysis and designing of digital systems.

A variable or function of variables in Boolean algebra can assume only 2 values, either 0 or 1. Hence there are no fraction, no negative number, no square root, no cube root, no logarithms, etc.

• Logical Operation

- Computer can perform only three kinds of operation.

- Addition of bits.
- Shifting of bits (left shift and right shift.)
- Complements of bits.



1. Logical addition:-OR

$$\begin{array}{l} 0 + 0 = 0 \\ 0 + 1 = 1 \\ 1 + 0 = 1 \\ 1 + 1 = 1 \end{array} \left\{ \begin{array}{l} \leftarrow \text{Can be compared with normal arithmetic} \\ \leftarrow \text{Boolean logic of addition of 2-bit is,} \\ \text{if any of the bit is 1, then result is 1.} \end{array} \right.$$

2. Logical Multiplication:- AND

$$\begin{array}{l} 0.0 = 0 \\ 0.1 = 0 \\ 1.0 = 0 \\ 1.1 = 1 \end{array} \left\{ \begin{array}{l} \text{Boolean logic of multiplication states that, when both bits} \\ \text{are '1', then result is '1', otherwise result is '0'}. \end{array} \right.$$

3. Complement:- NOT

$$\begin{array}{l} 0' = 1 \\ 1' = 0 \end{array}$$

4. Laws of Boolean Algebra

a. Properties of '0'

- i. $0 + p = p$
- ii. $0 \cdot p = 0$

b. Properties of '1'

- i. $1 + p = 1$
- ii. $1 \cdot p = p$

c. Absorption properties

- i. $p + pq = p$
- ii. $p + (p + q) = p + q$

d. Involution property

- i. $(p')' = p$

e. Idempotence property

- i. $p + p = p$
- ii. $p.p = p$

f. Complementarity Property

- i. $p + p' = 1$
- ii. $p.p' = 0$

g. Commutative property

- i. $p + q = q + p$
- ii. $p.q = q.p$

h. Associative property

- i. $(p + q) + r = p + (q + r)$
- ii. $(p.q).r = p.(q.r)$

i. Distributive property

- i. $p(q + r) = p.q + p.r$
- ii. $p + q.r = (p + q)(p + r)$
- iii. $p + p'q = (p + q)$

j. De'Morgan's law or Properties

- i. $(p + q)' = p'.q'$
- ii. $(p.q)' = p' + q'$

What is Truth Table?

A truth table is a mathematical table used in logic to represent all the combination of truth values for all variables involved in boolean expression.

0	p	q	$(p+q)'$	p'	q'	$p'.q'$	$(p.q)'$	$p'+q'$
1	0	0	1	1	1	1	1	1
2	0	1	0	1	0	0	1	1
3	1	0	0	0	1	0	1	1
4	1	1	0	0	0	0	0	0

↑ = ↑ 1st law ↑ = ↑ 2nd law Hence Proved

Proof of DeMorgan's theorem using Truth Table.

Logic gates

A gate is simply an electronic circuit which operates on one or more signals to produce an output signal. Gates are digital (2 - state) circuit because the input and output signals are either zero (low voltage) or one (high voltage) Gates are often called logic circuit because they can be analyzed with Boolean algebra.

Logic gates are of 3 types:

1. Basic gates

- i. NOT gate
- ii. OR Gate
- iii. AND Gate

2. Universal gates

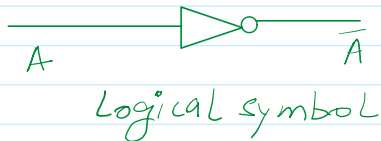
- NOR Gate or NOT(OR) Gate
- NAND Gate or NOT(AND) Gate

3. Exclusive gates

- XOR Gate or EXclusive OR Gate
- XNOR Gate or NOT(XOR) Gate

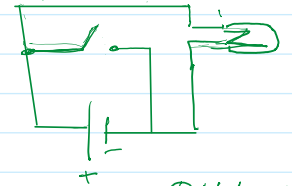
Basic Gates

- NOT Gate:** Is the gate with only one input signal and 1 inverted output signal or state is always opposite or barred.



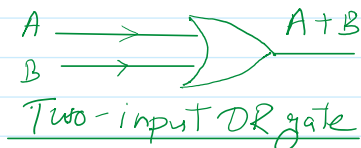
A	$\bar{A}$
0	1
1	0

Truth table



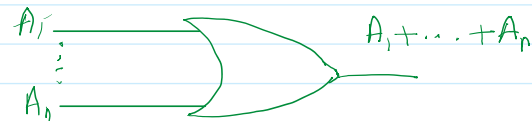
Electrical ckt.
ON $\leftarrow$ Not glow
OFF $\leftarrow$ glow

- OR Gate:** The OR gate has two or more input signals, but only one output signal. If the input signal is one or high or TRUE, the output signal is 1 or high or true. Otherwise it is 0 or low or false. It is represented with plus(+) symbol.

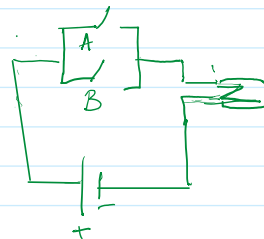


	A	B	A+B
0	0	0	0
1	0	1	1
2	1	0	1
3	1	1	1

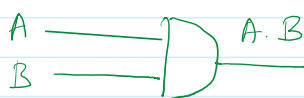
Truth table for OR gate 2-inputs & one output.



More than 2-input OR Gate

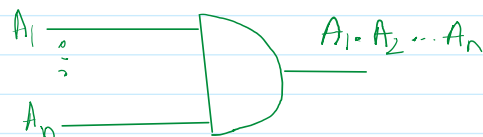


- AND Gate:** The AND gate can have two or more input signal and produce an output signal. When all the input signals are one or high or true then output signal is 1 or high or true, otherwise zero or low or false. It is Represented by dot '.' symbol.

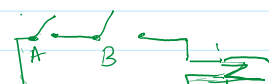


Two-input AND gate

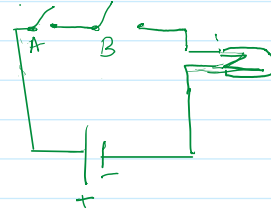
	A	B	A.B
0	0	0	0
1	0	1	0
2	1	0	0



N-input AND gate



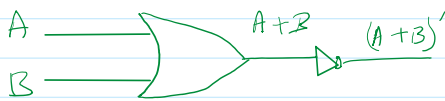
	A	B	A.B
1	0	0	0
2	0	1	0
3	1	0	0
4	1	1	1



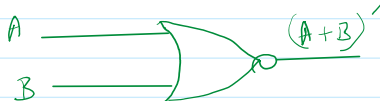
Truth table for AND gate 2-inputs & one output.

Universal Gates:

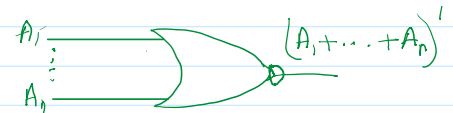
1. **NOR Gate:** In Nor gate we can input two or more signals, but we will get only one output signal. The output of NOR gate is inverted OR gate.



NOT(OR) gate
(two input)



NOR gate
(Two input)



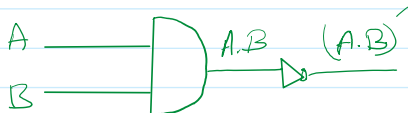
NOR gate
(N-input)

$$(A+B)' = A'B' \leftarrow \text{De Morgan's law.}$$

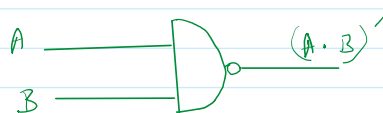
	A	B	A'	B'	(A+B)	A'B'
0	0	0	1	1	1	1
1	0	1	1	0	0	0
2	1	0	0	1	0	0
3	1	1	0	0	0	0

Truth table for 2-input NOR gate.

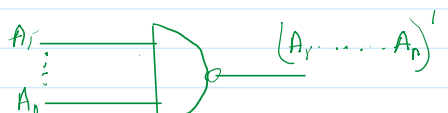
2. **NAND Gate:** In NAND gate we can input two or more signal but we can get only one output signal. In NAND gate we get the output of inverted AND gate.



NOT(AND) gate
(two input)



NAND gate
(Two input)



NAND gate
(N-input)

$$(A.B)' = A' + B' \leftarrow \text{De Morgan's law.}$$

	A	B	A'	B'	(A.B)	A'+B'
0	0	0	1	1	1	1
1	0	1	1	0	1	1
2	1	0	0	1	1	1
3	1	1	0	0	0	0

0	A	B	A'	B'	(A.B)'	A'+B'
1	0	0	1	1	1	1
2	0	1	1	0	1	1
3	1	0	0	1	1	1
4	1	1	0	0	0	0

Truth table for 2-input NAND gate.

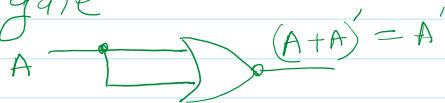
Why we use Universal Gates?

Ans: A universal gate is a logic gate which can implement any boolean function without using any other type of logic gate. The NOR gate and NAND gates are universal gates. This means you can create any logical boolean expression using only NOR gates or only NAND gates. We call it:

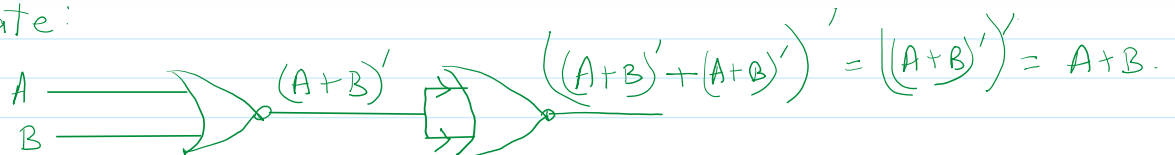
- NOR - to - NOR logic
- NAND - to - NAND logic

• NOR - to - NOR Logic

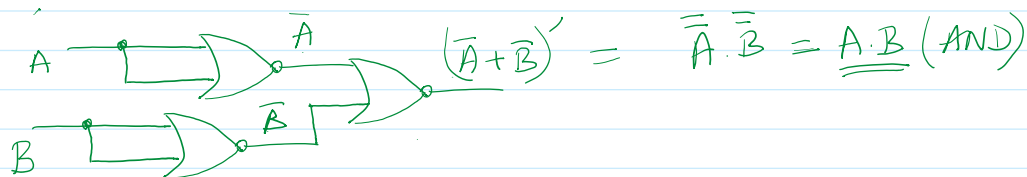
NOT gate



OR gate:



AND gate

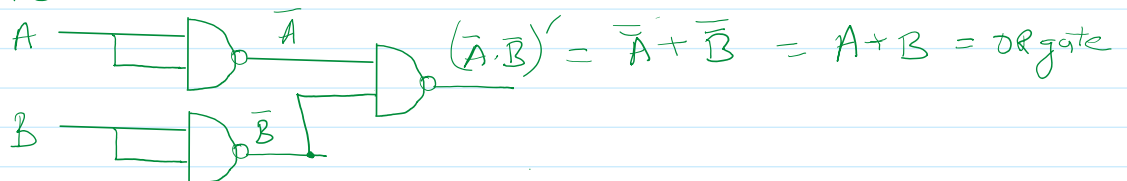


• NAND - to - NAND logic

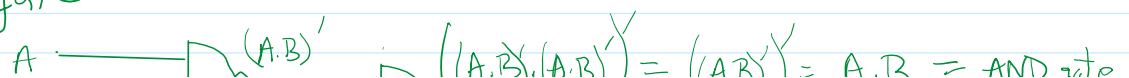
NOT Gate:



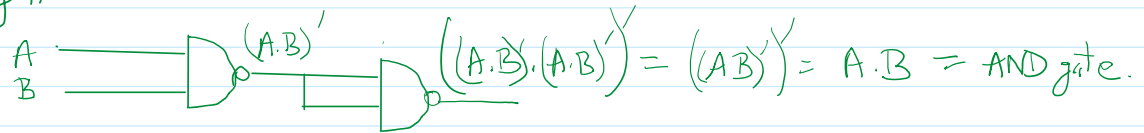
OR Gate:



AND gate:



NAND gate:

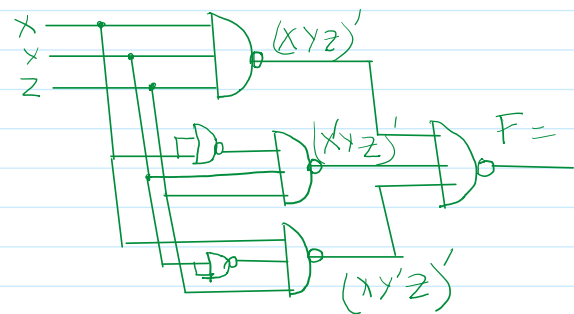
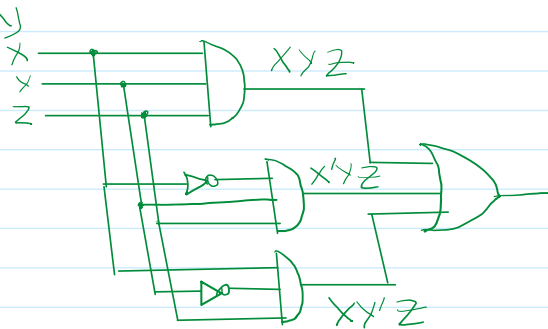


Design the circuit using NAND gates only or NOR gates only:

- Draw the expression using basic gates only.
- Replace all the gates with NAND gates or NOR gates depending upon a question.
- Try to resolve the expression, if you are getting the correct result, no need to add NAND gate or NOR gate to adjust the answer.
- If the expression is given in the form of SOP(Sum of Products), use NAND gates to draw the circuit, and
- If the expression is given in the form of POS(Product of Sum), use NOR gate to draw the circuit.

SOP expression $F(X, Y, Z) = XYZ + X'YZ + XY'Z$ [Use NAND Gate]

POS expression $F(X, Y, Z) = (X+Y+Z).(X'+Y+Z).(X+Y'+Z)$ [use NOR Gate]



$$\begin{aligned}
 &= [(XYZ)'.(X'YZ)'.(XY'Z)']' \\
 &= ((XYZ)')' + ((X'YZ)')' + ((XY'Z)')' \\
 &= XYZ + X'YZ + XY'Z
 \end{aligned}$$

